

SQL用語まとめ — 逆引きリファレンス

「こういうときは、どの句を使い、どんな書式だったか」を一目で引くためのカード集。(内容は原資料のまま)

キーワード

コマンド

関数

データ型

文字列・日付

可変部 (表・列名など)

演算子

■ 逆引きインデックス (やりたいこと → 使う句)

まず下の表で目的を探し、詳しい書式・例は各セクションのカードで確認できます。

やりたいこと	使う句・コマンド	書式の核
データベースを開く	sqlite3	sqlite3 データベース名
SQLファイルを実行する	.read	.read ファイル名
テーブルの一覧を見る	.tables	.tables
SQLiteを終了する	.exit	.exit
見出し・整形して表示	.header on / .mode column	.header on / .mode column
全カラムを取り出す	SELECT *	SELECT * FROM テーブル名;
特定のカラムを取り出す	SELECT カラム	SELECT カラム名 FROM テーブル名;
重複を消して取り出す	DISTINCT	SELECT DISTINCT カラム名 FROM テーブル名;
見出しに別名を付ける	AS	SELECT カラム名 AS 別名 FROM テーブル名;
条件で絞り込む	WHERE	SELECT * FROM テーブル名 WHERE 条件式;
文字列と一致で絞る	= と '...'	WHERE カラム名 = '値'
大小で絞る	比較演算子 < > <= >= <>	WHERE カラム名 >= 値
両方の条件を満たす	AND	WHERE 条件式1 AND 条件式2
どちらかの条件を満たす	OR	WHERE 条件式1 OR 条件式2
条件を満たさない	NOT	WHERE NOT 条件式
複数のどれかに一致	IN	WHERE カラム名 IN (データ1, データ2, ...)
複数のどれにも一致しない	NOT IN	WHERE カラム名 NOT IN (データ1, ...)
範囲で絞る	BETWEEN	WHERE カラム名 BETWEEN 値1 AND 値2
範囲外で絞る	NOT BETWEEN	WHERE カラム名 NOT BETWEEN 値1 AND 値2
NULLの行を探す	IS NULL	WHERE カラム名 IS NULL
件数/合計/平均/最小/最大	COUNT/SUM/AVG/MIN/MAX	SELECT SUM(カラム名) FROM テーブル名;
グループごとに集計	GROUP BY	SELECT カラム名, 関数(引数) FROM テーブル名 GROUP BY カラム名;
値をつないで表示	GROUP_CONCAT	GROUP_CONCAT(カラム名)
集計した結果に条件	HAVING	... GROUP BY カラム名 HAVING 条件式;

やりたいこと	使う句・コマンド	書式の核
並べ替える(降順/昇順)	ORDER BY (DESC/ASC)	... ORDER BY カラム名 DESC;
四捨五入する	ROUND	ROUND(数値, 桁)
行を追加する	INSERT INTO	INSERT INTO テーブル名 (...) VALUES (...);
値を更新する	UPDATE	UPDATE テーブル名 SET カラム名 = 値 WHERE 条件式;
行を削除する	DELETE	DELETE FROM テーブル名 WHERE 条件式;
テーブルを削除する	DROP TABLE	DROP TABLE テーブル名;
テーブル名を変える	ALTER ... RENAME TO	ALTER TABLE テーブル名 RENAME TO 新しいテーブル名;
テーブルを作る	CREATE TABLE	CREATE TABLE テーブル名 (カラム名 データ型 制約, ...);
2つのテーブルを結合	INNER JOIN ... ON	... INNER JOIN テーブル名2 ON 表1.列 = 表2.列
計算する	算術演算子 + - * / %	SELECT カラム名1 * カラム名2 FROM テーブル名;
バックアップ/復元	.backup / .restore	.backup ファイル名 / .restore ファイル名
CSVに書き出す	.mode csv / .once	.mode csv / .once ファイル名
コメントを書く	/* */	/* コメント */

1 データベースの基礎

データベース (DataBase)

保管されたデータの集まりと、そのデータを管理するシステムのこと。

例) ショッピングサイトの商品情報や顧客情報などがデータベースで管理されている。大量のデータの中から目的のデータを迅速に取り出せる。

DBMS (DataBase Management System)

コンピュータ上でデータの管理を行うソフトウェア。データベース管理システムを略してDBMSという。

<種類> ・階層型データベース / ・ネットワーク型データベース / ・リレーショナルデータベース

RDBMS

リレーショナルデータベースの管理を行うDBMSのこと。RはRelational（関係のある）の略。

例）大規模なシステム向けから小規模なシステム向けまで、さまざまな種類がある。

【代表的なRDBMS】

名前	説明
Oracle Database (オラクルデータベース)	商用でのシェアが高く、大規模なシステムで利用される
MySQL (マイエスキューエル)	オープンソースで、Webアプリケーションの開発に利用される
PostgreSQL (ポストグレスキューエル)	拡張性があり独自の処理を追加できることから、複雑なデータ構造のシステムに利用される
SQLite (エスキューライト)	小規模で高速に動作することから、小規模なシステムやIoT機器（電子機器）などに利用される

リレーショナルデータベースの構造

データをテーブルと呼ばれる2次元の表形式で表す。列にあたる部分をカラム、行にあたる部分をレコード、レコード内の個々のデータが入る場所をフィールドと呼ぶ。

SQL

RDBMSでデータを操作するためのデータベース言語。SQLで書いた「SQL文」という命令で、データを入れたり取り出したりできる。

【SQLiteの特徴】

メリット	デメリット
・軽量なため、動作が高速 ・複雑な設定が不要 ・導入が容易	・複数人で同時に操作できない ・パスワード機能がないため、セキュリティ面に不安がある ・何十万件もデータがある大規模なデータベースには不向き

2 SQLite操作の基本（コマンド）

カレントディレクトリを変更する

書式

`cd` フォルダのパス

SQLiteを起動する

書式

```
sqlite3 開きたいデータベース名
```

例

```
sqlite3 sample.db
```

ファイルを読み込む

書式

```
.read ファイル名
```

例

```
.read chap1_sales.sql
```

テーブルの一覧を表示

書式

```
.tables
```

SQLiteを終了する

書式

```
.exit
```

ヘッダー（見出し）を表示する

書式

```
.header on
```

表示モードをカラムにする

書式

```
.mode column
```

見やすく整形して表示する（組み合わせ例）

```
.header on  
.mode column  
SELECT * FROM sales;
```

3 SELECT — データを取り出す

テーブルから全カラムを取り出す

書式

```
SELECT * FROM テーブル名;
```

例

```
SELECT * FROM sales;
```

「*（アスタリスク）」は全カラムを取り出す指定。「FROM」のあとに操作するテーブル名を書く。

SQL文の終わりには「;（セミコロン）」を付ける決まり。付け忘れるとエラーになる。

指定したカラムのデータを取り出す

書式

```
SELECT カラム名 FROM テーブル名;
```

例

```
SELECT customer FROM sales;
```

重複した値を取り除く

書式

```
SELECT DISTINCT カラム名 FROM テーブル名;
```

例

```
SELECT DISTINCT customer FROM sales;
```

複数のカラムを取り出す

カラム名は「, (カンマ)」で区切ると複数指定でき、指定した順番に取り出せる。

書式

```
SELECT カラム名1, カラム名2, ... FROM テーブル名;
```

例

```
SELECT date, item_name, item_count FROM sales;
```

見出しに別名（エイリアス）を付ける

ヘッダーに表示されるカラム名などに一時的な別名を付けられる。カラム名のあとに「AS 別名」と書く。

書式

```
SELECT カラム名 AS 別名 FROM テーブル名;
```

例

```
SELECT item_name AS 商品名, item_count AS 個数 FROM sales;
```

4 WHERE — 条件で絞り込む

条件を指定してデータを取り出す

条件を指定するときは FROM 句のあとに WHERE 句で条件を指定する。

書式

```
SELECT * FROM テーブル名 WHERE 条件式;
```

例

```
SELECT * FROM sales WHERE item_name = 'リンゴ';
```

条件式に使う値が文字列の場合は「' (シングルクォーテーション)」で囲む。

「= (イコール)」は比較演算子。演算子の左右にある値を比較する。

【比較演算子】

記号	意味
=	右辺と左辺が同じ
<	左辺が右辺より小さい
>	左辺が右辺より大きい
<=	左辺が右辺以下
>=	左辺が右辺以上
<>	左辺と右辺が違う

大小を比較して絞る（比較演算子の例）

```
item_count > 2      -- 2より大きい（2は含まない）
item_count < 2      -- 2より小さい（2は含まない）
item_count >= 2     -- 2以上（2を含む）
item_count <= 2     -- 2以下（2を含む）
```

日付で絞る

日付は「yyyy-mm-dd」の形式で表す。

例

```
SELECT * FROM sales WHERE date >= '2023-04-11';  -- 4/11以降
SELECT * FROM sales WHERE date > '2023-04-11';   -- 4/11よりあと
```

両方の条件を満たす（かつ）

左右どちらも満たすデータを取り出すときは「かつ」の意味の AND 演算子を使う。

書式

```
WHERE 条件式1 AND 条件式2
```

例

```
SELECT * FROM sales
WHERE customer = 'サル橋' AND item_name = 'バナナ';
```

どちらかの条件を満たす（もしくは）

どちらかを満たすデータを取り出すときは「もしくは」の意味の OR 演算子を使う。

書式

```
WHERE 条件式1 OR 条件式2
```

例

```
SELECT * FROM sales  
WHERE item_name = 'リンゴ' OR item_name = 'ブドウ';
```

条件を満たさない（ではない）

条件を満たさないデータを取り出すときは「ではない」の意味の NOT 演算子を使う。

書式

```
WHERE NOT 条件式
```

例

```
SELECT * FROM sales WHERE NOT item_name = 'リンゴ';
```

演算子の優先順位

高	<>>=<= / =<>
↓	NOT
↓	AND
低	OR

例

```
SELECT * FROM sales  
WHERE customer = 'サル橋' OR customer = 'ウサ田'  
AND item_name = 'リンゴ';
```

※ AND 演算子から先に処理される。

優先順位の低い演算子から処理したい場合は、その部分をカッコで囲む。

例

```
SELECT * FROM sales  
WHERE (customer = 'サル橋' OR customer = 'ウサ田')  
AND item_name = 'リンゴ';
```

※ カッコで囲んだため、OR 演算子から処理される。

複数のどれかに一致（IN）

複数の値のいずれかに一致するものを取り出すときは IN 演算子が便利。

書式

```
WHERE カラム名 IN (データ1, データ2, ...)
```

例

```
SELECT * FROM sales WHERE item_name IN ('リンゴ', 'イチゴ');
```

「=」と「OR」で書くと： WHERE item_name = 'リンゴ' OR item_name = 'イチゴ';

複数のどれにも一致しない (NOT IN)

いずれの値にも一致しないデータを取り出すときは NOT IN演算子を使う。

書式

```
WHERE カラム名 NOT IN (データ1, データ2, ...);
```

例

```
SELECT * FROM sales WHERE item_name NOT IN ('リンゴ', 'イチゴ');
```

<>とANDで書くと： WHERE item_name <> 'リンゴ' AND item_name <> 'イチゴ';

範囲で絞る (BETWEEN)

数や日付で特定の範囲を指定したいときは BETWEEN演算子を使う。

書式

```
WHERE カラム名 BETWEEN 値1 AND 値2;
```

例

```
SELECT * FROM sales  
WHERE date BETWEEN '2023-04-11' AND '2023-04-13';
```

※ BETWEENと組み合わせるANDは「かつ」ではなく「から」の意味になる。

比較演算子で書くと： WHERE date >= '2023-04-11' AND date <= '2023-04-13';

範囲外で絞る (NOT BETWEEN)

指定した範囲外のデータを取り出したいときは NOT BETWEEN演算子を使う。

書式

```
WHERE カラム名 NOT BETWEEN 値1 AND 値2;
```

例

```
SELECT * FROM sales  
WHERE date NOT BETWEEN '2023-04-11' AND '2023-04-13';
```

比較演算子で書くと： WHERE date < '2023-04-11' OR date > '2023-04-13';

5 集計・グループ化

集計関数

対象テーブルのカラムの合計値や平均値など、数値から何らかの結果を返す関数。関数に渡すデータや条件を引数、得られる結果を戻り値という。

【集計関数】

関数	意味
COUNT (カウント)	テーブルのレコード数を求める
SUM (サム)	指定したカラムの合計値を求める
AVG (エイブイジー)	指定したカラムの平均値を求める
MIN (ミン)	指定したカラムの最小値を求める
MAX (マックス)	指定したカラムの最大値を求める

関数を使う基本のSQL文

書式

```
SELECT 関数名(引数) FROM テーブル名;
```

レコード数を数える

例

```
SELECT COUNT(*) FROM sales;
```

ASで別名を付けられる： SELECT COUNT(*) AS records FROM sales;

合計・平均を求める

例

```
SELECT SUM(price) FROM sales;    -- 合計
SELECT AVG(price) FROM sales;    -- 平均
```

最小値・最大値を求める

例

```
SELECT MIN(price), MAX(price) FROM sales;
```

※ 関数も「,」で区切ると同時に使える。(原文の綴りママ)

グループごとに集計する (GROUP BY)

データをグループ化するには「グループ分けする」意味の GROUP BY句を使う。

書式

```
SELECT カラム名, 関数(引数) FROM テーブル名 GROUP BY カラム名;
```

例

```
SELECT customer, COUNT(*) FROM sales GROUP BY customer;
```

※ SELECT句とGROUP BY句で同じカラム名を指定しなければならない。

例

```
SELECT item_name, SUM(price) FROM sales GROUP BY item_name;  
SELECT date, AVG(price) FROM sales GROUP BY date;
```

指定したカラムの値を結合する (GROUP_CONCAT)

例

```
SELECT customer, GROUP_CONCAT(item_name) FROM sales  
GROUP BY customer;
```

引数にDISTINCTを付けると重複を除ける: GROUP_CONCAT(DISTINCT カラム名)

例

```
SELECT customer, GROUP_CONCAT(DISTINCT item_name) FROM sales  
GROUP BY customer;
```

集計した結果に条件を付ける (HAVING)

集計結果に対して条件を付けたい場合は HAVING句を使う。

書式

```
SELECT カラム名, 集計関数 FROM テーブル名  
GROUP BY カラム名 HAVING 条件式;
```

例

```
SELECT customer, COUNT(*) FROM sales  
GROUP BY customer HAVING COUNT(*) >= 2;
```

5つの句を使ったSQL文

例

```
SELECT customer, COUNT(*) FROM sales  
WHERE customer <> 'ウサ田'  
GROUP BY customer HAVING COUNT(*) >= 2;
```

FROM句

→

WHERE句

→

GROUP BY句

→

HAVING句

→

SELECT句

6 ORDER BY — 並べ替える

取り出したデータを並べ替える

並べ替えたいときは ORDER BY句を使う。

書式

```
SELECT * FROM テーブル名 ORDER BY 並べ替え基準のカラム名;
```

例

```
SELECT * FROM sales ORDER BY price;
```

降順に並べ替える (DESC)

例

```
SELECT * FROM sales ORDER BY price DESC;
```

※ DESCはDescendingの略で降順の意味。書かない場合は自動的に昇順 (ASC) として実行される。

複数の基準で並べ替える

書式

```
SELECT * FROM テーブル名  
ORDER BY 並べ替え基準のカラム名1 DESC, 並べ替え基準のカラム名2 DESC;
```

例

```
SELECT * FROM sales ORDER BY item_count DESC, price DESC;
```

WHERE句と組み合わせる

書式

```
SELECT * FROM テーブル名 WHERE 条件式 ORDER BY カラム名;
```

例

```
SELECT * FROM sales WHERE item_name = 'バナナ' ORDER BY price;
```

GROUP BY句と組み合わせる

書式

```
SELECT カラム名1, 集計関数 FROM テーブル名  
GROUP BY カラム名1 [HAVING 条件式]  
ORDER BY 並べ替え基準のカラム名もしくは集計関数;
```

例

```
SELECT item_name, SUM(item_count) FROM sales  
GROUP BY item_name  
ORDER BY SUM(item_count) DESC;
```

6つの句を使ったSQL文

例

```
SELECT item_name, SUM(item_count) FROM sales  
WHERE item_name <> 'バナナ'  
GROUP BY item_name HAVING SUM(item_count) > 2  
ORDER BY SUM(item_count) DESC;
```

FROM句

→

WHERE句

→

GROUP BY句

→

HAVING句

→

SELECT句

→

ORDER BY句

CRUD（クラッド）

DBMSの基本機能 Create（作成）・Read（読み込み）・Update（更新）・Delete（削除）の頭文字。

・読み込み=SELECT文 / ・作成=CREATE文・INSERT文 / ・更新=UPDATE文 / ・削除=DELETE文

バックアップファイルを作る

書式 `.backup` 作成するファイル名

例 `.backup sample.back`

バックアップから復元する

書式 `.restore` バックアップファイル名

データを追加する①（カラム名を指定）

書式 `INSERT INTO` テーブル名 (カラム名1, カラム名2, ...)
`VALUES` (値1, 値2, ...);

例 `INSERT INTO sales (date, item_name, item_count, price, customer)`
`VALUES ('2023-04-15', 'イチゴ', 2, 800, 'クマ井');`

データを追加する②（カラム名を省略）

書式 `INSERT INTO` テーブル名 `VALUES` (値1, 値2, ...);

例 `INSERT INTO sales`
`VALUES ('2023-04-15', 'イチゴ', 1, 400, 'ウサ田');`

値が足りないINSERT文（注意例）

-- ① カラム名を指定・値が1つ足りない → NULL 値のセルができる

`INSERT INTO sales (date, item_name, item_count, price)`
`VALUES ('2023-04-16', 'ブドウ', 1, 500);`

-- ② カラム名を省略・値が足りない → Parseエラー

`INSERT INTO sales VALUES ('2023-04-16', 'バナナ', 2, 400);`

※ あえてNULLを入れる場合以外は、カラム名を省略したINSERT文にすると値の過不足ミスが起きにくい。

NULL値があるときのCOUNT

例

```
SELECT COUNT(*), COUNT(customer) FROM sales;
```

※ 「*」は全レコード数を数えるが、カラム名を引数にするとNULL値以外の値が入ったレコード数を数える。

NULL値を持つデータを探す (IS NULL)

例

```
SELECT date, item_name, item_count, price  
FROM sales WHERE customer IS NULL;
```

※ IS NULL演算子でNULL値を持つデータを探せる。

コメントを書く

```
/*salesテーブルからデータを取り出す*/  
SELECT * FROM sales;
```

※ /* から */ の範囲はコメントで、SQL文の一部とは見なされない。

実行前の状態へ復元する

```
.backup sample2.back  
.restore sample.back
```

データを更新する (UPDATE)

書式

```
UPDATE テーブル名 SET カラム名 = 値 WHERE 条件式;
```

例

```
UPDATE sales SET date = '2023-04-15' WHERE date = '2023-04-14';
```

複数の値をまとめて更新する

書式

```
UPDATE テーブル名 SET カラム名1 = 値1, カラム名2 = 値2 WHERE 条件式;
```

例

```
UPDATE sales SET item_count = 1, price = 400  
WHERE item_name = 'イチゴ' AND customer = 'ネコ村';
```

※ WHERE句で条件を指定しないと、SET句で指定したカラムの値がすべて同じになってしまう。

データを削除する (DELETE)

書式

```
DELETE FROM テーブル名 WHERE 条件式;
```

例

```
DELETE FROM sales WHERE date = '2023-04-10';
```

※ WHERE句を使わないと、すべてのレコードが削除される。

テーブルを削除する (DROP TABLE)

書式

```
DROP TABLE テーブル名;
```

例

```
DROP TABLE sales;
```

8 テーブル設計 — 命名規則・CREATE・制約

命名規則

テーブル名やカラム名を付けるときのルール。

- ・半角のアルファベット、数字、_（アンダースコア）を使う / ・大文字か小文字のどちらかに統一する / ・名前はアルファベットではじめ、単語は「_」で区切る / ・テーブル名やカラム名の重複はNG / ・キーワードとの重複を避ける

テーブル名を変更する (ALTER)

既存のテーブル名を変更するときは ALTER文を使う。

書式

```
ALTER TABLE テーブル名 RENAME TO 新しいテーブル名;
```

例

```
ALTER TABLE sales RENAME TO old_sales;
```

テーブルを作成する（CREATE）

データベースにテーブルを作るには CREATE文を使う。

書式

```
CREATE TABLE テーブル名 (  
  カラム名1 データ型 制約,  
  カラム名2 データ型 制約,  
  ...  
);
```

【データ型の種類】

型	意味
NULL	何もデータが入っていないことを表すNULL値のこと
TEXT	文字列
INTEGER	整数
REAL	浮動小数点数
BLOB	Binary Large Objectの略で、バイナリデータのこと

制約

テーブルに設定するルールのようなもの。データの整合性や一貫性を維持するために使う。

主キー制約

各レコードを一意に識別するためのルール。主キーのカラムは値の重複とNULL値が禁止される。

書式

```
カラム名 データ型 PRIMARY KEY
```

NOT NULL制約

設定されたカラムはNULL値が禁止され、必ずデータが入っていることが保証される。

書式

```
カラム名 データ型 NOT NULL
```

新しいテーブルを作る（例①・②）

例

```
CREATE TABLE sales (  
  date TEXT NOT NULL,  
  item_id INTEGER NOT NULL,  
  item_count INTEGER NOT NULL,  
  customer TEXT NOT NULL  
);
```

例

```
CREATE TABLE item (  
  item_id INTEGER PRIMARY KEY,  
  item_name TEXT NOT NULL,  
  price INTEGER NOT NULL,  
  farm TEXT NOT NULL,  
  date TEXT NOT NULL  
);
```

9 INNER JOIN — テーブルを結合する

テーブルの内部結合の基本

テーブルを結合するときは「内部結合」の意味の INNER JOIN句を使う。

書式

```
SELECT カラム名 FROM テーブル名1  
INNER JOIN テーブル名2 ON テーブル名1.カラム名 = テーブル名2.カラム名;
```

例

```
SELECT * FROM sales  
INNER JOIN item ON sales.item_id = item.item_id;
```

結合したテーブルからカラムを個別指定

例

```
SELECT sales.date, sales.item_id, item_name, item_count, customer  
FROM sales INNER JOIN item ON sales.item_id = item.item_id;
```

結合して条件を指定する

書式

```
SELECT カラム名1, カラム名2, ... FROM テーブル名1  
INNER JOIN テーブル名2 ON テーブル名1.カラム名 = テーブル名2.カラム名  
WHERE 条件式;
```

例

```
SELECT sales.date, sales.item_id, item_name, item_count, customer  
FROM sales INNER JOIN item ON sales.item_id = item.item_id  
WHERE item_name = 'リンゴ';
```

FROM句 → INNER JOIN句 ON句 → WHERE句 → SELECT句

結合したテーブルをグループ化する

例

```
SELECT sales.item_id, item_name, SUM(item_count)
FROM sales INNER JOIN item ON sales.item_id = item.item_id
GROUP BY sales.item_id;
```

参考：itemテーブル（結合の相手）

商品ID	商品名	価格
1	イチゴ	400
2	リンゴ	200
3	ブドウ	300
4	モモ	350

10 計算 — 算術演算子

算術演算・算術演算子

コンピュータ上で行う数値の計算を算術演算といい、計算に使う記号を算術演算子という。

【算術演算子】

演算子	意味
+	足し算
-	引き算
*	掛け算
/	割り算
%	割り算の余りを求める

算術演算子を使った計算

例

```
SELECT 5 + 2, 5 - 2, 5 * 2, 5 / 2, 5 % 2;
```

テーブルのデータを使って計算する

例

```
SELECT sales.date, sales.item_id, item_name, item_count * price
FROM sales INNER JOIN item ON sales.item_id = item.item_id;
```

別名を付ける：item_count * price AS total_price

関数の引数に式を渡す

書式

関数名(カラム名1 算術演算子 カラム名2)

例

```
SELECT SUM(item_count * price)
FROM sales INNER JOIN item ON sales.item_id = item.item_id;
```

関数の結果を使って計算する

書式

関数名(カラム名1) 算術演算子 カラム名2

例

```
SELECT sales.item_id, item_name, SUM(item_count) * price
FROM sales INNER JOIN item
ON sales.item_id = item.item_id
GROUP BY sales.item_id
ORDER BY SUM(item_count) * price DESC;
```

四捨五入する (ROUND)

例

```
SELECT ROUND(5.678, 2);
```

11 CSVファイルへの出力

CSV形式の表示に変更する

書式

.mode csv

SQL文の結果をファイルに出力する

書式

.once ファイル名

データをCSVファイルに書き出す (組み合わせ例)

```
.mode csv
.once export_sample.csv
SELECT sales.date, sales.item_id, item_name, item_count, customer
FROM sales INNER JOIN item ON sales.item_id = item.item_id;
```